BÀI TẬP CSP VÀ GAME TRONG AI

Họ và tên	Võ Thị Như Ý
MSSV / Lớp	3123410439/DTN1241

Mục tiêu học tập

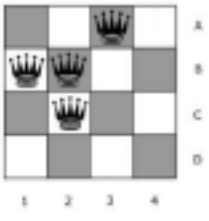
• Mô hình hóa và giải các bài toán thỏa mãn ràng buộc (CSP).
• Vận dụng Forward Checking, AC-3, MRV và LCV trong CSP.
• Phân tích Minimax, Alpha-Beta Pruning và Expectimax trong trò chơi đối kháng.
• Nhận biết ảnh hưởng của thứ tự duyệt và cắt tỉa đến hiệu quả tìm kiếm.
• Vận dụng các kỹ thuật trên vào bài toán 4-Queens, điều phối đường băng và Magic Square.

Nội dung	Kỹ thuật	Ý nghĩa chính
CSP	Forward Checking	Loại giá trị xung đột sau mỗi phép gán
CSP	AC-3	Duy trì tính nhất quán cung
CSP	MRV / LCV	Chọn biến khó trước / chọn giá trị ít ràng buộc nhất
Game	Minimax / Alpha-Beta	Tìm quyết định tối ưu và cắt tỉa nhánh không ảnh hưởng
Game	Expectimax	Mô hình hóa nút ngẫu nhiên bằng kỳ vọng

Bài 1. Bài toán 4-Queens và Hill Climbing

Hình dưới trình bày giả mã thuật toán hill-climbing cho bài toán 4 quân hậu và một trạng thái khởi đầu. Các quân hậu di chuyển như trong cờ vua: có thể đi nhiều ô theo hàng, cột hoặc đường chéo miễn là đường đi không bị chặn.

```
function 4-QUEENS-HILL-CLIMBING(problem)
    current_state = problem.INITIAL-STATE
    loop do
        neighbor = state with lowest number of conflicts between queens obtained by
                   making one legal chess move from current_state
        if neighbor.num_conflicts >= current_state.num_conflicts
            return current_state
        current_state = neighbor
```



Hình 1. Mã giả Hill Climbing và trạng thái khởi đầu của bài toán 4-Queens

a) Hãy ghi trạng thái của từng quân hậu sau một vòng lặp của thuật toán. Một vòng lặp tương đương với việc di chuyển một quân hậu.

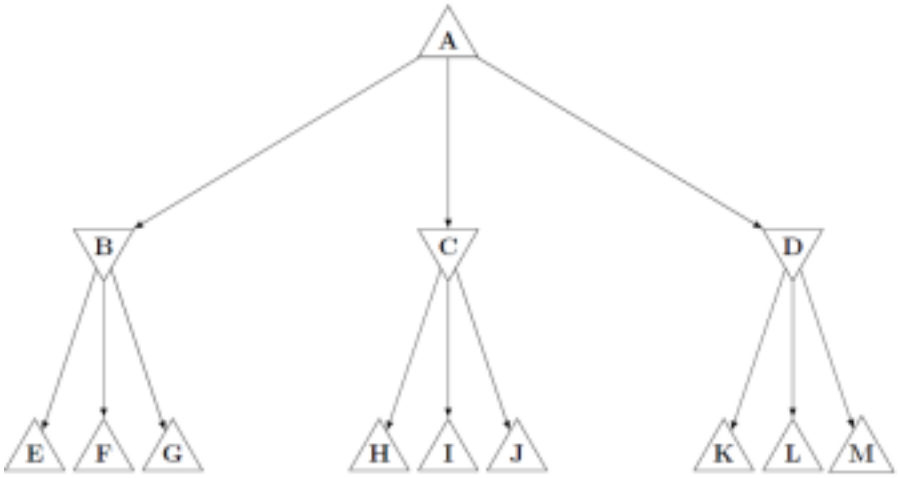
Trạng thái ban đầu có 4 cặp quân hậu xung đột. Di chuyển quân hậu tại B2 đến D4, số cặp xung đột giảm còn 1.
[A3,B1,C2,D4]

b) Thuật toán hill-climbing ở trên có dừng với trạng thái khởi đầu đã cho hay không? Nếu có, trạng thái cuối thuộc loại nào? Chọn tất cả phương án phù hợp.

☐ A) Không dừng
☒ B) Cực trị cục bộ (local optimum)
☐ C) Cực trị toàn cục (global optimum)
☐ D) Vai bằng (shoulder)
☐ E) Không thuộc các trường hợp trên
Ở trạng thái $A3, B1, C2, D4$, vẫn còn một cặp quân hậu xung đột. Không có nước di chuyển hợp lệ nào làm số xung đột giảm xuống dưới 1, nên thuật toán dừng tại cực trị cục bộ, không phải cực trị toàn cục.

Bài 2. Alpha-Beta Pruning kết hợp Iterative Deepening

Xét cây trò chơi trong Hình 2. Gốc A là nút MAX; B, C, D là các nút MIN. Bảng bên phải cho giá trị hàm đánh giá f(s) của các trạng thái. Iterative deepening có thể kết hợp với cắt tỉa để tăng khả năng pruning và giảm thời gian tìm kiếm.



Evaluation Function	
State, s	$f(s)$
B	4
C	14
D	19
E	5
F	7
G	8
H	17
I	16
J	15
K	23
L	20
M	26

Hình 2. Cây trò chơi và bảng giá trị hàm đánh giá

Thực hiện theo hai bước: (1) sắp thứ tự các cây con B, C, D dựa trên f(B), f(C), f(D), từ tốt nhất đến xấu nhất theo góc nhìn của nút cha; (2) áp dụng Alpha-Beta Pruning theo thứ tự mới, giới hạn độ sâu đến các nút cháu E, F, G, ..., và dùng giá trị đánh giá tại các nút cháu.

a) Thứ tự các cây con sau khi sắp xếp là:

☐ B → C → D
☐ C → D → B

☒ D → C → B

b) Với thứ tự mới, hãy liệt kê các nút KHÔNG được thăm do bị cắt tỉa. Các nút cháu trong mỗi cây con vẫn được xét từ trái sang phải.

I,J,F,G

c) Alpha-Beta Pruning chuẩn đảm bảo cho cùng nước đi với Minimax đầy đủ. Phương pháp iterative deepening kết hợp pruning như mô tả ở trên có luôn đảm bảo trả về cùng nước đi với Minimax trên toàn bộ cây hay không? Giải thích ngắn gọn.

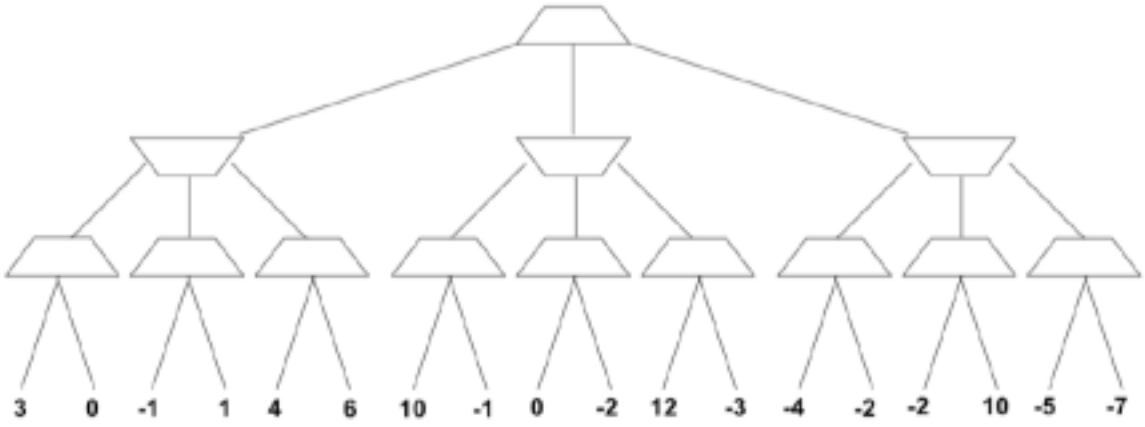
☐ Có

☒ Không

Do giới hạn độ sâu, thuật toán sử dụng giá trị đánh giá tại các nút cháu thay vì xét toàn bộ cây. Giá trị đánh giá có thể khác với giá trị Minimax thực tế, nên không đảm bảo luôn chọn cùng nước đi với Minimax trên toàn bộ cây.

Bài 3. Adversarial Search - Alpha-Beta Pruning

Xét cây trò chơi dưới đây, trong đó nút gốc là MAX. Áp dụng Alpha-Beta Pruning và duyệt các nút kế tiếp từ trái sang phải.



Hình 3. Cây trò chơi dùng cho Alpha-Beta Pruning

Yêu cầu:

• Ghi giá trị α và β tại mỗi nút trong quá trình duyệt.

• Ghi giá trị được trả về tại mỗi nút trung gian.

• Đánh dấu X lên các cạnh hoặc nhánh bị cắt tỉa.

• Cho biết giá trị Minimax tại nút gốc và nước đi tối ưu của MAX.

Lời giải:
Duyệt các nút từ trái sang phải.
Cây con bên trái:
Nút MAX thứ nhất: $\max(3, 0) = 3$.

Nút MAX thứ hai: $\max(-1, 1) = 1$.

Nút MAX thứ ba: $\max(4, 6) = 6$.

Nút MIN bên trái: $\min(3, 1, 6) = 1$.

Cây con ở giữa:
Nút MAX thứ nhất: $\max(10, -1) = 10$.

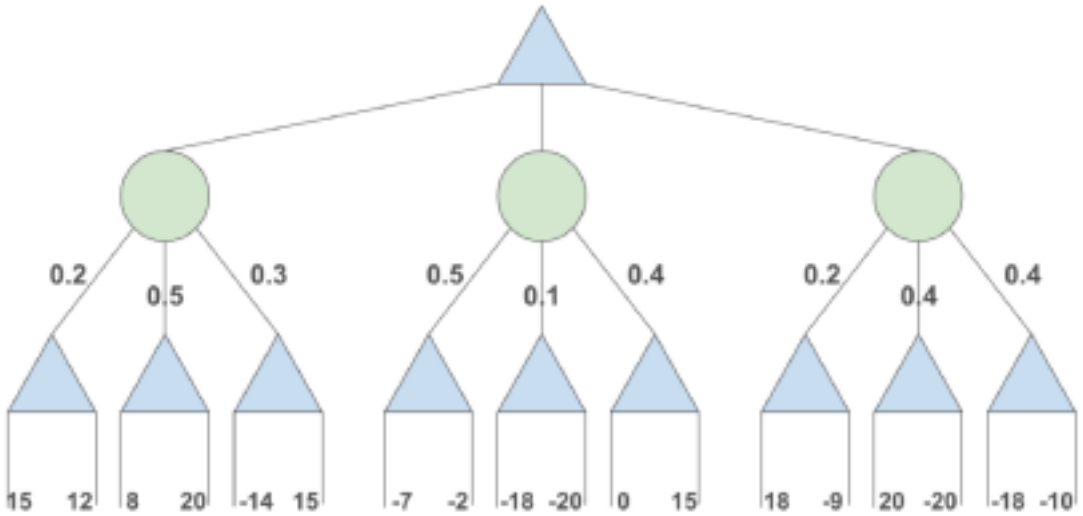
Nút MAX thứ hai: $\max(0, -2) = 0$.

Vì $0 \leq \alpha = 1$, cắt tỉa toàn bộ nút MAX thứ ba, gồm hai lá $12, -3$.
Nút MIN ở giữa: 0.
Cây con bên phải:
Nút MAX thứ nhất: $\max(-4, -2) = -2$.

Vì $-2 \leq \alpha = 1$, cắt tỉa hai nút MAX còn lại, gồm các lá $(-2, 10)$ và $(-5, -7)$.
Nút MIN bên phải: ≤ -2.
Giá trị tại nút gốc MAX: $max(1, 0, \leq -2) = 1$
Nước đi tối ưu của MAX: Chọn cây con bên trái.
Giá trị α, β tại các nút:
Gốc MAX: ban đầu $(\alpha, \beta) = (-\infty, +\infty)$, kết thúc $(1, +\infty)$.
MIN bên trái: kết thúc $(-\infty, 1)$.
MIN ở giữa: kết thúc $(1, 0)$, cắt tỉa nhánh còn lại khi $0 \leq 1$.
MIN bên phải: sau nút MAX đầu tiên có $\beta = -2$, cắt tỉa các nhánh còn lại khi $-2 \leq 1$.

Bài 4. Expectimax và Chance Nodes

Xét cây trò chơi trong Hình 4. Nút gốc là MAX (tam giác màu xanh), các nút tròn màu xanh lá là chance node. Xác suất của mỗi nhánh được ghi trên cạnh. Các nút MAX ở tầng dưới chọn giá trị lớn nhất trong các lá của chúng.



Hình 4. Cây Expectimax với chance nodes

a) Duyệt các nút con từ trái sang phải. Hãy tính giá trị được trả về tại từng nút MAX, từng chance node và tại nút gốc.

Chance node bên trái:

M1 = max(15, 12) = 15

M2 = max(8, 20) = 20

M3 = max(-14, 15) = 15

C1 = 0,2 × 15 + 0,5 × 20 + 0,3 × 15

C1 = 3 + 10 + 4,5 = 17,5

Chance node ở giữa:

M4 = max(-7, -2) = -2

M5 = max(-18, -20) = -18

M6 = max(0, 15) = 15

C2 = 0,5 × (-2) + 0,1 × (-18) + 0,4 × 15

C2 = -1 - 1,8 + 6 = 3,2

Chance node bên phải:

M7 = max(18, -9) = 18

M8 = max(20, -20) = 20

M9 = max(-18, -10) = -10

C3 = 0,2 × 18 + 0,4 × 20 + 0,4 × (-10)

C3 = 3,6 + 8 - 4 = 7,6

Giá trị tại nút gốc MAX:

V(A) = max(17,5; 3,2; 7,6)

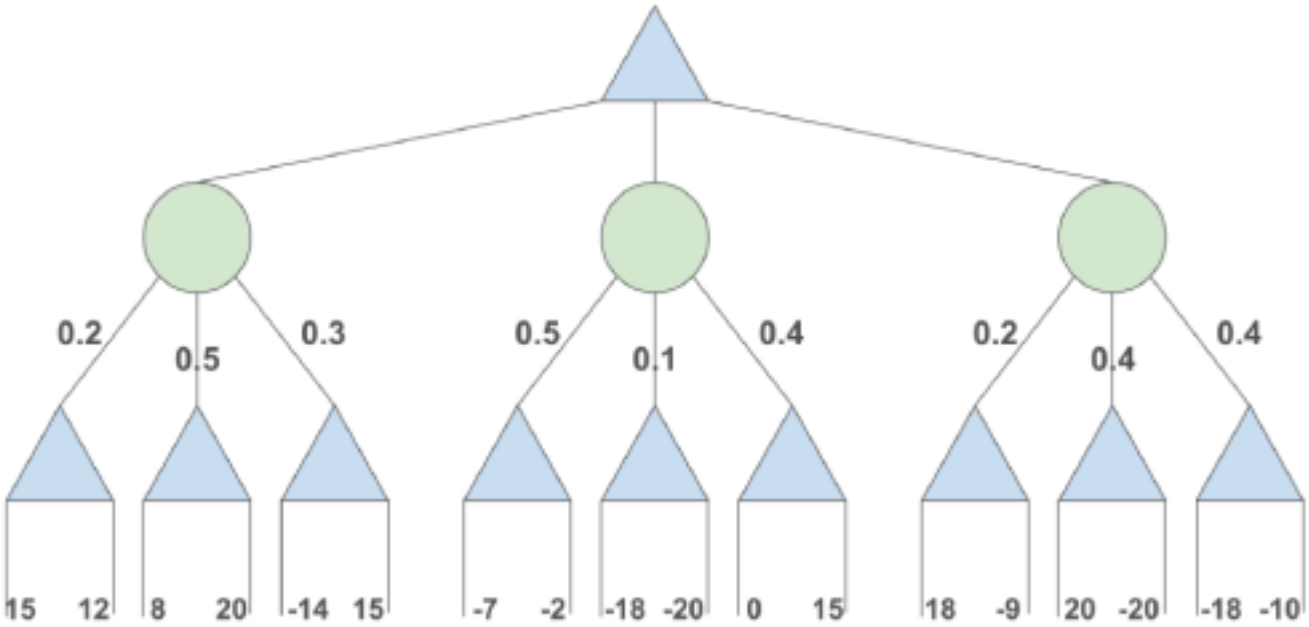
V(A) = 17,5

b) Nếu chưa biết thêm giới hạn nào về giá trị lá, những nút nào trong cây trên có thể được cắt tỉa mà vẫn đảm bảo kết quả chính xác? Giải thích.

Không thể cắt tỉa chắc chắn các nhánh chưa xét của chance node chỉ dựa trên những giá trị đã quan sát, vì giá trị kỳ vọng còn phụ thuộc vào các nhánh chưa xét.

Các nút MAX cũng không thể cắt tỉa an toàn nếu không có giới hạn phù hợp cho giá trị các lá còn lại.

c) Giả sử mỗi nút lá đều có giá trị nằm trong đoạn [-20, 20]. Hãy giải lại cây và cắt tỉa các nhánh không cần thiết; vẫn duyệt từ trái sang phải.



Hình 5. Cây Expectimax dùng cho câu 4(c)

Chance node bên trái được tính đầy đủ:

C1 = 17,5

Tại nút gốc MAX, giá trị tốt nhất hiện tại là 17,5.

Chance node ở giữa:

Tính hai nhánh đầu:

$S = 0,5 \times (-2) + 0,1 \times (-18)$

$S = -1 - 1,8 = -2,8$

Giá trị lớn nhất có thể của nhánh còn lại:

$U = -2,8 + 0,4 \times 20$

$U = 5,2$

Vì 5,2 nhỏ hơn 17,5 nên cắt tỉa nút MAX thứ ba cùng hai lá 0 và 15.

Chance node bên phải:

Tiếp tục duyệt từ trái sang phải. Các nhánh được xét đầy đủ vì giới hạn trên của phần chưa xét chưa đủ để kết luận giá trị chance node không thể vượt 17,5.

$C3 = 0,2 \times 18 + 0,4 \times 20 + 0,4 \times (-10)$

$C3 = 3,6 + 8 - 4 = 7,6$

Kết quả cuối cùng:

$V(A) = \max(17,5; 3,2; 7,6)$

$V(A) = 17,5$

Nút MAX tối ưu là nút con bên trái.

Bài 5. Câu hỏi thảo luận về CSP

a) Forward Checking khác AC-3 ở điểm nào? So sánh phạm vi lan truyền ràng buộc, thời điểm thực hiện và chi phí tính toán.

Forward Checking kiểm tra miền giá trị của các biến chưa gán có liên quan trực tiếp đến biến vừa được gán. Thuật toán được thực hiện sau mỗi phép gán, có chi phí tính toán thấp hơn nhưng phạm vi lan truyền ràng buộc hạn chế.

AC-3 duy trì tính nhất quán cung bằng cách kiểm tra các cung và loại bỏ những giá trị không có giá trị hỗ trợ trong miền của biến liên quan. Khi miền thay đổi, các cung liên quan được kiểm tra lại. AC-3 có khả năng lan truyền ràng buộc rộng hơn nhưng thường tốn nhiều chi phí tính toán hơn.

b) Vì sao các heuristic sau thường được dùng trong bài toán CSP?

i) Minimum Remaining Values (MRV)

MRV chọn biến có số giá trị hợp lệ còn lại ít nhất. Heuristic này ưu tiên giải quyết biến khó trước, giúp phát hiện mâu thuẫn sớm và giảm số lần quay lui.

ii) Least Constraining Value (LCV)

LCV chọn giá trị loại bỏ ít lựa chọn nhất của các biến chưa gán có liên quan. Heuristic này giúp duy trì nhiều lựa chọn cho các bước tiếp theo, tăng khả năng tìm được nghiệm.

Bài 6. CSP - Điều phối không lưu

Có năm máy bay A, B, C, D và E cùng hai đường băng: quốc tế và nội địa. Cần xếp cho mỗi máy bay một khung giờ và một đường băng để hạ cánh hoặc cất cánh. Mỗi đường băng có bốn khung giờ 1, 2, 3, 4. Bài toán phải thỏa các ràng buộc sau:
• Máy bay B bị hỏng một động cơ và phải hạ cánh ở khung giờ 1.
• Máy bay D chỉ có thể đến sân bay để hạ cánh từ khung giờ 3 trở đi.
• Máy bay A sắp hết nhiên liệu và chỉ có thể chờ tối đa đến khung giờ 2.
• Máy bay D phải hạ cánh trước khi máy bay C cất cánh vì có hành khách nối chuyến từ D sang C. • Không có hai máy bay nào được đặt cùng một khung giờ trên cùng một đường băng.

a) Hãy mô hình hóa bài toán dưới dạng CSP: xác định biến, miền giá trị và các ràng buộc đơn/nguyên (unary) và nhị phân (binary). Viết ràng buộc bằng ký hiệu toán học hoặc logic. Bổ sung hai ràng buộc sau:

• A, B, C là các chuyến bay quốc tế và chỉ dùng đường băng quốc tế.
• D, E là các chuyến bay nội địa và chỉ dùng đường băng nội địa.

Tập biến:

X =

Trong đó mỗi biến biểu diễn khung giờ của một máy bay.

Miền giá trị:

D(A) = D(B) = D(C) = D(D) = D(E) = 4

Ràng buộc đơn (Unary):

Máy bay B phải hạ cánh ở khung giờ 1: B = 1.

Máy bay D chỉ được hạ cánh từ khung giờ 3 trở đi: D $\in$ 4.

Máy bay A chỉ có thể chờ tối đa đến khung giờ 2: A $\in$ 2.

Ràng buộc nhị phân (Binary):

Máy bay D phải hạ cánh trước máy bay C cất cánh: $D < C$.

Hai máy bay dùng cùng đường băng không được ở cùng khung giờ:

Nếu A, B, C cùng dùng đường băng quốc tế thì $A \neq B$, $A \neq C$, $B \neq C$.

Nếu D, E cùng dùng đường băng nội địa thì $D \neq E$.

b) Với hai ràng buộc bổ sung, loại đường băng không cần nằm trong miền giá trị vì đã được xác định bởi biến. Hãy xác định miền mới và vẽ đồ thị ràng buộc của bài toán.

Do đường băng đã được xác định:

A, B, C dùng đường băng quốc tế.

D, E dùng đường băng nội địa.

Miền giá trị sau khi áp dụng ràng buộc đơn:

$A = 2$

$B = 1$

$C = 4$

$D = 4$

$E = 4$

Đồ thị ràng buộc:

Các đỉnh: A, B, C, D, E.

Các cạnh: AB, AC, BC, CD, DE.

*c) Sau khi áp dụng arc consistency, miền của các biến còn lại là gì? Bắt đầu bằng việc áp dụng các ràng buộc unary. Hãy gạch bỏ những giá trị bị loại khỏi miền. *

Từ $B = 1$ và $A \neq B$, loại 1 khỏi miền A:

$A = 2$

Từ $A = 2$, $B = 1$ và các ràng buộc khác nhau trên đường băng quốc tế:

$C = 4$

Từ $D < C$, với $D = 4$ và $C = 4$:

$D = 3$

$C = 4$

Do D = 3 và D ≠ E:

E = 4

Kết quả:

A = 2

B = 1

C = 4

D = 3

E = 4

d) Thay vì duy trì arc consistency toàn cục, hãy dùng Forward Checking kết hợp Backtracking Search. Chọn biến theo MRV; khi hòa, chọn ký tự nhỏ hơn trước; chọn giá trị nhỏ hơn trước. Hãy liệt kê các cặp (biến, giá trị) theo đúng thứ tự phát sinh, kể cả các phép gán bị hoàn tác khi gặp dead end. Áp dụng ràng buộc unary trước khi bắt đầu.

A	1	2	3	4
B	1	2	3	4
C	1	2	3	4
D	1	2	3	4
E	1	2	3	4

Áp dụng ràng buộc đơn trước khi bắt đầu.

Thứ tự gán biến theo MRV; nếu hòa, chọn ký tự nhỏ hơn trước. Giá trị được chọn theo thứ tự tăng dần.

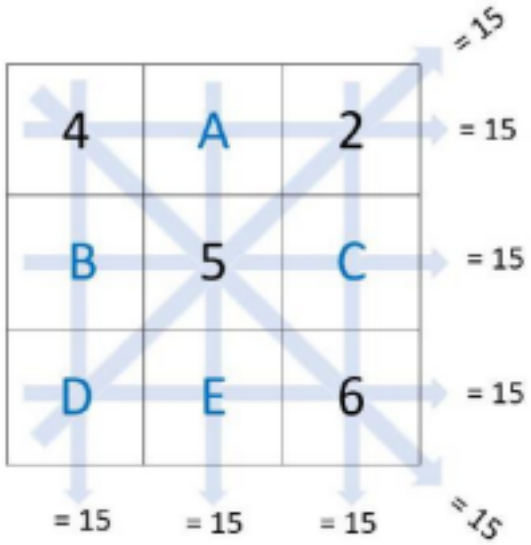
1. Gán (B, 1).
2. Gán (A, 2).
3. Gán (C, 3).
4. Gán (D, 4).
5. Phát hiện dead end vì D = 4 không thỏa mãn D < C.
6. Hoàn tác (D, 4).
7. Hoàn tác (C, 3).
8. Gán (C, 4).
9. Gán (D, 3).
10. Gán (E, 1).

Nghiệm cuối cùng:

A = 2, B = 1, C = 4, D = 3, E = 1.

Bài 7. CSP - Magic Square

Magic Square là bảng n × n trong đó mỗi ô chứa một giá trị khác nhau thuộc {1, ..., n^2}; tổng của mỗi hàng, mỗi cột và hai đường chéo đều bằng nhau. Với bảng 3 × 3 dưới đây, tổng cần đạt là 15. Một số ô đã được cho trước, các ô ký hiệu chữ cái cần được xác định.



Hình 6. Bài toán Magic Square 3 × 3

a) Hãy mô hình hóa bài toán dưới dạng CSP: biến, miền giá trị và các ràng buộc. Có thể bỏ qua việc tạo biến cho các ô đã biết.

Tập biến:

X =

Miền giá trị:

D(A) = D(B) = D(C) = D(D) = D(E) = 9

Các ràng buộc:

Mỗi ô chứa một giá trị khác nhau:

AllDiff(4, A, 2, B, 5, C, D, E, 6)

Tổng mỗi hàng bằng 15:

4 + A + 2 = 15

B + 5 + C = 15

D + E + 6 = 15

Tổng mỗi cột bằng 15:

4 + B + D = 15

A + 5 + E = 15

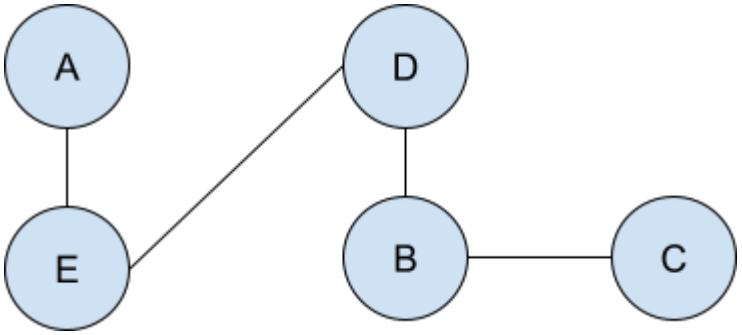
2 + C + 6 = 15

Tổng hai đường chéo bằng 15:

4 + 5 + 6 = 15

2 + 5 + D = 15

b) Vẽ đồ thị ràng buộc nhị phân. Để đơn giản, có thể bỏ qua ràng buộc alldiff khi vẽ đồ thị.



c) Sử dụng đồ thị ràng buộc nhị phân và thuật toán AC-3 để thu hẹp miền, sau đó tìm một nghiệm cho Magic Square.

Từ ràng buộc hàng thứ nhất:

4 + A + 2 = 15

A = 9

Từ ràng buộc cột thứ ba:

2 + C + 6 = 15

C = 7

Từ ràng buộc đường chéo:

2 + 5 + D = 15

D = 8

Từ ràng buộc cột thứ nhất:

4 + B + D = 15

4 + B + 8 = 15

B = 3

Từ ràng buộc cột thứ hai:

A + 5 + E = 15

9 + 5 + E = 15

E = 1

Nghiệm cuối cùng:

A = 9, B = 3, C = 7, D = 8, E = 1.

Bảng Magic Square hoàn chỉnh:

4	**9**	**2**
3	5	7
8	1	6

Kiểm tra: Tổng mỗi hàng, mỗi cột và hai đường chéo đều bằng 15.

Bài 8. MRV và LCV trong thực hành

Ba sinh viên AJ, Sunny và Ellyse cần chọn mỗi người đúng một màu sơn sao cho không có hai người chọn cùng màu. Các tập màu có sẵn:

• AJ = {Đỏ, Vàng, Xanh lá}
• Sunny = {Đỏ, Vàng, Hồng}
• Ellyse = {Đỏ, Hồng}

a) Dùng MRV để chọn biến và LCV để chọn giá trị. Nếu hòa, phá hòa theo thứ tự cầu vồng. Hỏi thuật toán phải backtrack về phép gán trước đó bao nhiêu lần? Hãy ghi lại thứ tự gán.

Bước 1: MRV chọn Ellyse vì có ít giá trị nhất.

LCV so sánh:

Đỏ ảnh hưởng đến cả AJ và Sunny.

Hồng chỉ ảnh hưởng đến Sunny.

Chọn Hồng: $Ellyse = Hồng$

Bước 2: Sunny còn . Hai giá trị có mức độ ràng buộc bằng nhau với AJ, nên phá hòa theo thứ tự cầu vồng: chọn Đỏ. $Sunny =$ Đỏ

Bước 3: AJ còn , chọn Vàng theo thứ tự cầu vồng. $AJ = Vàng$

Nghiệm:

Sinh viên	**Màu**
Ellyse	Hồng
Sunny	Đỏ
AJ	Vàng

Số lần backtrack: 0.

b) Thay MRV bằng Maximum Remaining Values và thay LCV bằng Most Constraining Value. Hỏi số lần backtrack là bao nhiêu? So sánh với câu (a) và giải thích vì sao.

Bước 1: AJ và Sunny đều có 3 giá trị. Phá hòa theo thứ tự tên, chọn AJ.

Bước 2: Chọn giá trị ràng buộc nhiều nhất là Đỏ vì ảnh hưởng đến cả Sunny và Ellyse. $AJ =$ Đỏ

Khi đó:

Sunny = Vàng, Hồng

Ellyse = Hồng

Bước 3: Chọn Sunny vì có nhiều giá trị còn lại hơn Ellyse. Theo Most Constraining Value, thử Hồng trước vì nó ảnh hưởng đến Ellyse.

$$Sunny = Hồng$$

Ellyse không còn màu hợp lệ, nên phải quay lui một lần.

Thử lại:

$$Sunny = Vàng, Ellyse = Hồng$$

Nghiệm cuối:

Sinh viên	Màu
AJ	Đỏ
Sunny	Vàng
Ellyse	Hồng

Số lần backtrack: 1.

So sánh: câu a cần 0 lần quay lui, câu b cần 1 lần vì cách chọn giá trị ràng buộc nhiều nhất có thể tạo ngõ cụt sớm.

--- Hết ---